Lab Programs (71-89)

K.Dhilip

192365013

1.

2.

3.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
192365013RD
QUESTIONS 20 / 20 completed
Upper
UPPER
Write a Java program to convert a given string to uppercase.
PROGRAM EDITOR
Java
Run
Save
1 public class Main {
2 public static void main(String[] args) {
3 String str = "Hello World";
4
5 String result = str.toUpperCase();
6
7 System.out.println("Uppercase: " + result);
8 }
9 }
INPUT
Your INPUT goes here!
OUTPUT
Uppercase: HELLO WORLD

4.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
192365013RD
QUESTIONS 20 / 20 completed
RM-Char
RM-CHAR
Write a Java program to remove a specified character from a given string.
PROGRAM EDITOR
Java
Run
Save
1 public class Main {
2 public static void main(String[] args) {
3 String str = "Hello World";
4 char ch = 'l';
5
6 String result = str.replace(String.valueOf(ch), "");
7
8 System.out.println("Original: " + str);
9 System.out.println("After removing '" + ch + "': " + result);
10 }
11 }
INPUT
Your INPUT goes here!
OUTPUT
Original: Hello World
After removing 'l': Heo Word

5.

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip 192365013RD

QUESTIONS 20 / 20 completed

Replace

REPLACE

Write a Java program to replace a specified character with another character in a given string.

PROGRAM EDITOR — Java — Run — Save

```java
public class Main {
    public static void main(String[] args) {
        String str = "Hello World";

        String result = str.replace('l', 'x');

        System.out.println("Original: " + str);
        System.out.println("After replacement: " + result);
    }
}
```

INPUT

Your INPUT goes here!

OUTPUT

```
Original: Hello World
After replacement: Hexxo Worxd
```

6.

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip 192365013RD

QUESTIONS 20 / 20 completed

Split

SPLIT

Write a Java program to split a given string into an array of substrings based on a specified delimiter.

PROGRAM EDITOR — Java — Run — Save

```java
public class Main {
    public static void main(String[] args) {
        String str = "Apple,Banana,Orange";

        String[] parts = str.split(",");

        for (String s : parts)
            System.out.println(s);
    }
}
```

INPUT

Your INPUT goes here!

OUTPUT

```
Apple
Banana
Orange
```

7.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
192365013RD
QUESTIONS 20 / 20 completed
Trim
TRIM
Write a Java program to trim a given string and remove leading and trailing spaces.
PROGRAM EDITOR
Java
Run
Save
1 public class Main {
2 public static void main(String[] args) {
3 String str = " Hello World ";
4
5 String result = str.trim();
6
7 System.out.println("Original: [" + str + "]");
8 System.out.println("Trimmed: [" + result + "]");
9 }
10 }
INPUT
Your INPUT goes here!
OUTPUT
Original: [Hello World]
Trimmed: [Hello World]

8.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
192365013RD
QUESTIONS 20 / 20 completed
Convert
CONVERT
Write a Java program to convert a given string to a char array.
PROGRAM EDITOR
Java
Run
Save
1 public class Main {
2 public static void main(String[] args) {
3 String str = "Hello";
4
5 char[] arr = str.toCharArray();
6
7 for (char c : arr)
8 System.out.println(c);
9 }
10 }
INPUT
Your INPUT goes here!
OUTPUT
H
e
l
l
o

9.

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip 192365013RD

QUESTIONS 2 / 2 completed

Sahred

SAHRED

Write a Java program to demonstrate how to use synchronization to prevent multiple threads from accessing a shared resource simultaneously.

PROGRAM EDITOR — Java — Run — Save

```
public class Main {
    public static void main(String[] args) {
        Counter c = new Counter();

        Thread t1 = new Thread(c);
        Thread t2 = new Thread(c);

        t1.start();
        t2.start();
    }
}

class Counter implements Runnable {
    int count = 0;

    public synchronized void run() {
        for (int i = 1; i <= 5; i++) {
            count++;
            System.out.println(Thread.currentThread().getName() +
                               " : " + count);
        }
```

INPUT

Your INPUT goes here!

OUTPUT

```
Thread-0 : 1
Thread-0 : 2
Thread-0 : 3
Thread-0 : 4
Thread-0 : 5
Thread-1 : 6
Thread-1 : 7
Thread-1 : 8
Thread-1 : 9
Thread-1 : 10
```

10.

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip 192365013RD

QUESTIONS 2 / 2 completed

AtomicInteger

ATOMICINTEGER

Write a Java program to demonstrate how to use the AtomicInteger class to provide atomic access to a shared integer variable.

Output:
Shared value: 20000

This confirms that the two threads were able to increment the shared value atomically and the final value of the shared value is as

PROGRAM EDITOR — Java — Run — Save

```
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        AtomicInteger value = new AtomicInteger(0);

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 10000; i++)
                value.incrementAndGet();
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 10000; i++)
                value.incrementAndGet();
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();
```

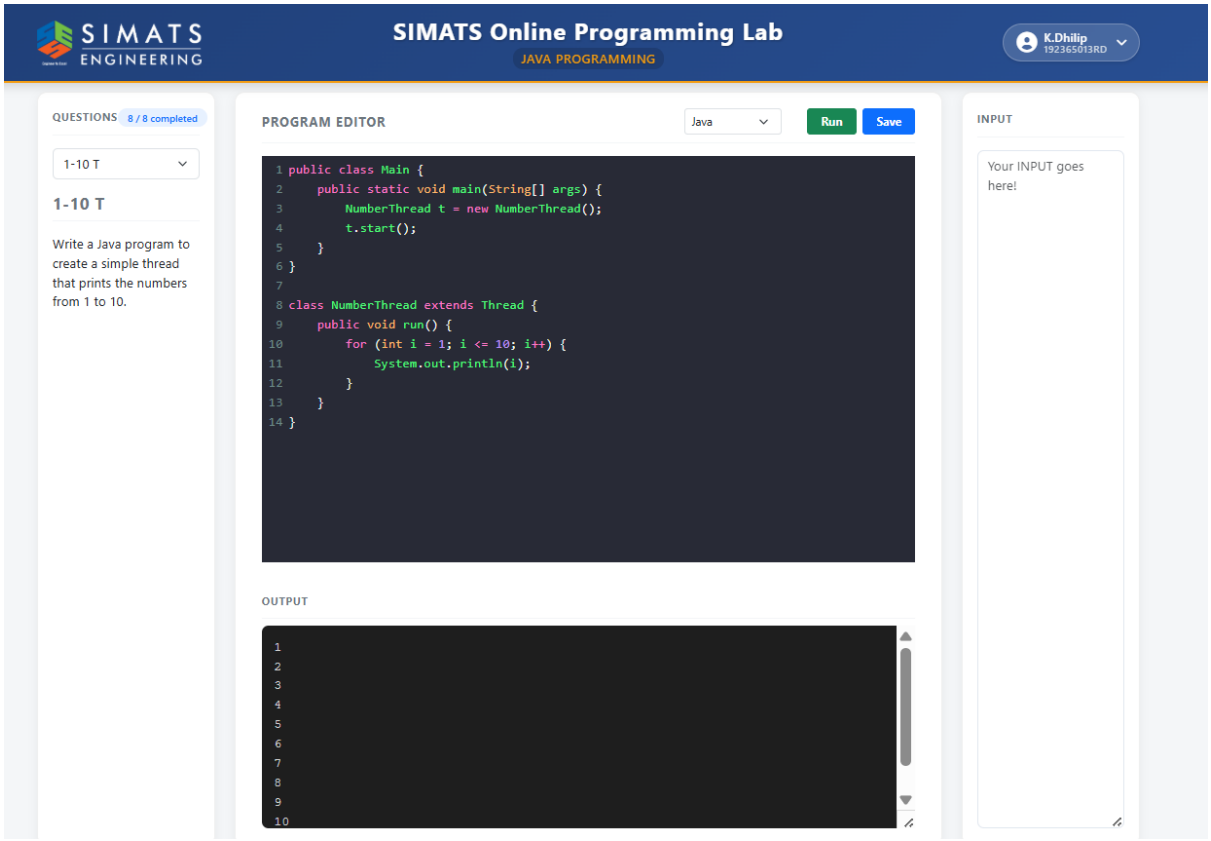
INPUT

Your INPUT goes here!

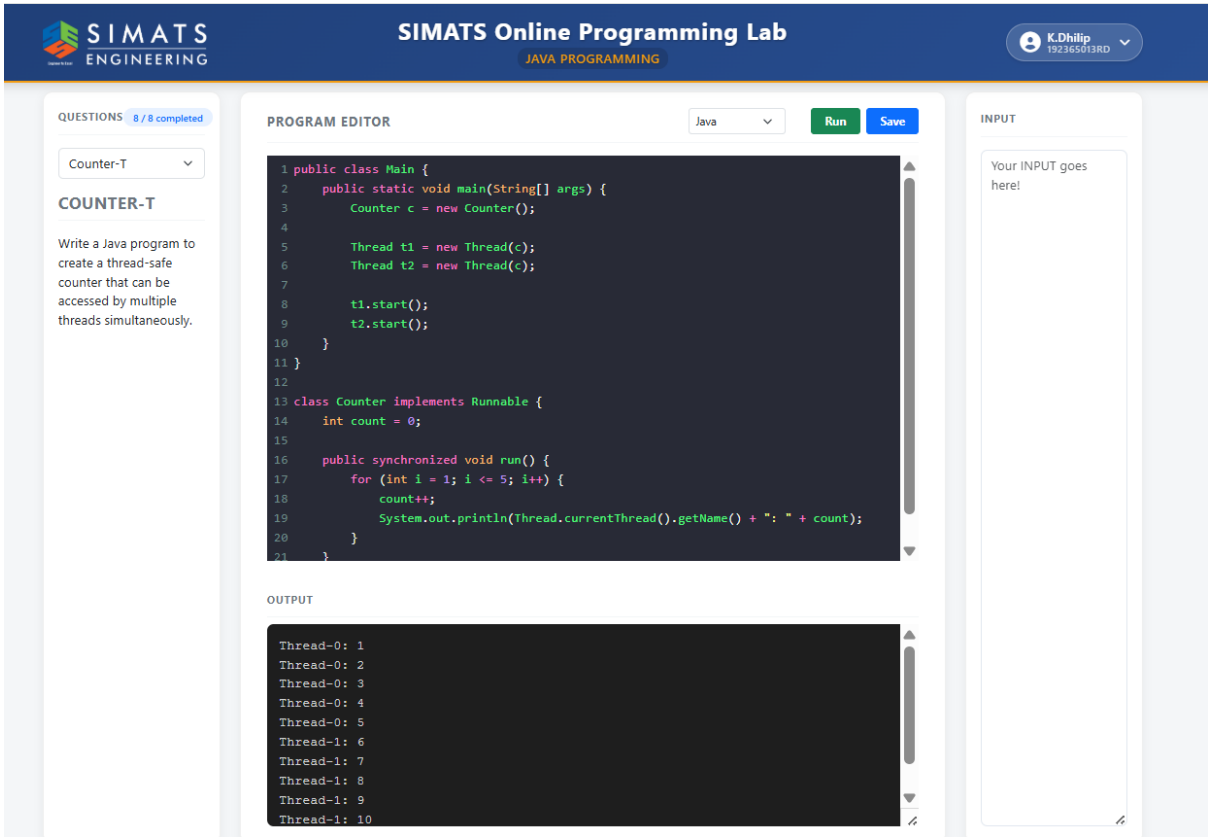
OUTPUT

```
Shared value: 20000
```

11.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
192365013RD
QUESTIONS 8 / 8 completed
1-10 T
1-10 T
Write a Java program to create a simple thread that prints the numbers from 1 to 10.
PROGRAM EDITOR
Java
Run
Save
1 public class Main {
2 public static void main(String[] args) {
3 NumberThread t = new NumberThread();
4 t.start();
5 }
6 }
7
8 class NumberThread extends Thread {
9 public void run() {
10 for (int i = 1; i <= 10; i++) {
11 System.out.println(i);
12 }
13 }
14 }
OUTPUT
1
2
3
4
5
6
7
8
9
10
INPUT
Your INPUT goes here!

12.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
192365013RD
QUESTIONS 8 / 8 completed
Counter-T
COUNTER-T
Write a Java program to create a thread-safe counter that can be accessed by multiple threads simultaneously.
PROGRAM EDITOR
Java
Run
Save
1 public class Main {
2 public static void main(String[] args) {
3 Counter c = new Counter();
4
5 Thread t1 = new Thread(c);
6 Thread t2 = new Thread(c);
7
8 t1.start();
9 t2.start();
10 }
11 }
12
13 class Counter implements Runnable {
14 int count = 0;
15
16 public synchronized void run() {
17 for (int i = 1; i <= 5; i++) {
18 count++;
19 System.out.println(Thread.currentThread().getName() + ": " + count);
20 }
21 }
OUTPUT
Thread-0: 1
Thread-0: 2
Thread-0: 3
Thread-0: 4
Thread-0: 5
Thread-1: 6
Thread-1: 7
Thread-1: 8
Thread-1: 9
Thread-1: 10
INPUT
Your INPUT goes here!

13.

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip 192365013RD

QUESTIONS 8 / 8 completed

M-Thread

M-THREAD

Write a Java program to create multiple threads that perform some time-consuming task, such as calculating prime numbers or sorting an array, and compare their execution times.

PROGRAM EDITOR — Java — Run — Save

```java
public class Main {
    public static void main(String[] args) throws Exception {
        Thread t1 = new PrimeThread(1, 50000);
        Thread t2 = new PrimeThread(50001, 100000);

        long start = System.currentTimeMillis();

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        long end = System.currentTimeMillis();

        System.out.println("Execution Time: " + (end - start) + " ms");
    }
}

class PrimeThread extends Thread {
    int start, end;
```

INPUT

Your INPUT goes here!

OUTPUT

```
Execution Time: 781 ms
```

14.

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip 192365013RD

QUESTIONS 8 / 8 completed

Pro-Consume

PRO-CONSUME

Write a Java program to create a producer-consumer scenario using multiple threads. The producer thread should add items to a queue, and the consumer thread should remove items from the queue and process them.

PROGRAM EDITOR — Java — Run — Save

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();

        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                synchronized (q) {
                    q.add(i);
                    System.out.println("Produced: " + i);
                    q.notify();
                }
            }
        });

        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                synchronized (q) {
                    while (q.isEmpty()) {
                        try {
```

INPUT

Your INPUT goes here!

OUTPUT

```
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
```

15.

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip 192365013RD

QUESTIONS 8 / 8 completed

Deadlock

DEADLOCK

Write a Java program to create a deadlock scenario using multiple threads.

PROGRAM EDITOR — Java — Run — Save

```
public class Main {
    public static void main(String[] args) throws Exception {
        Object a = new Object();
        Object b = new Object();

        Thread t1 = new Thread(() -> {
            synchronized (a) {
                System.out.println("Thread 1 locked A");
                try { Thread.sleep(1000); } catch (Exception e) {}

                synchronized (b) {
                    System.out.println("Thread 1 locked B");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (b) {
                System.out.println("Thread 2 locked B");
                try { Thread.sleep(1000); } catch (Exception e) {}

```

INPUT

Your INPUT goes here!

OUTPUT

```
Thread 2 locked B
Thread 1 locked A
Deadlock created!
```

16.

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip 192365013RD

QUESTIONS 8 / 8 completed

Wait-Notify

WAIT-NOTIFY

Write a Java program to demonstrate how to use the "wait" and "notify" methods to implement a producer-consumer scenario.

PROGRAM EDITOR — Java — Run — Save

```
public class Main {
    public static void main(String[] args) {
        Box box = new Box();

        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 5; i++)
                box.produce(i);
        });

        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 5; i++)
                box.consume();
        });

        producer.start();
        consumer.start();
    }
}

class Box {
    int item;
```

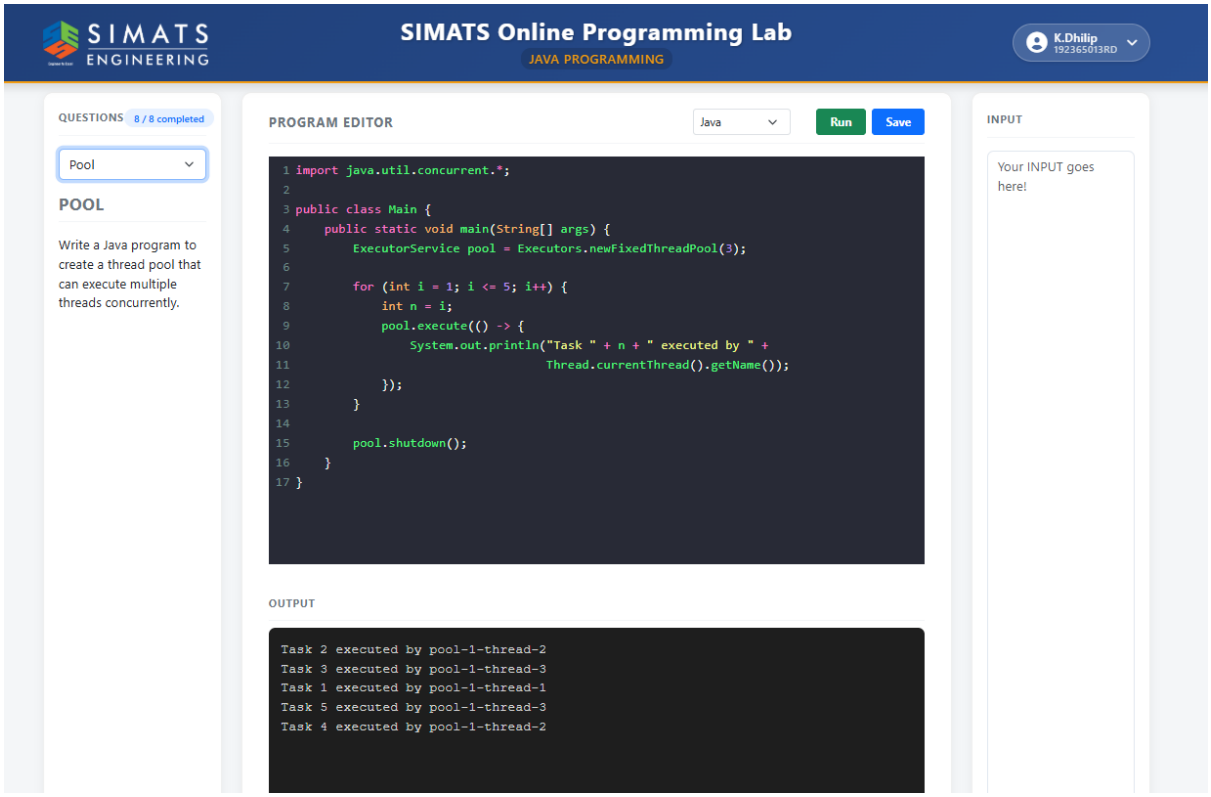
INPUT

Your INPUT goes here!

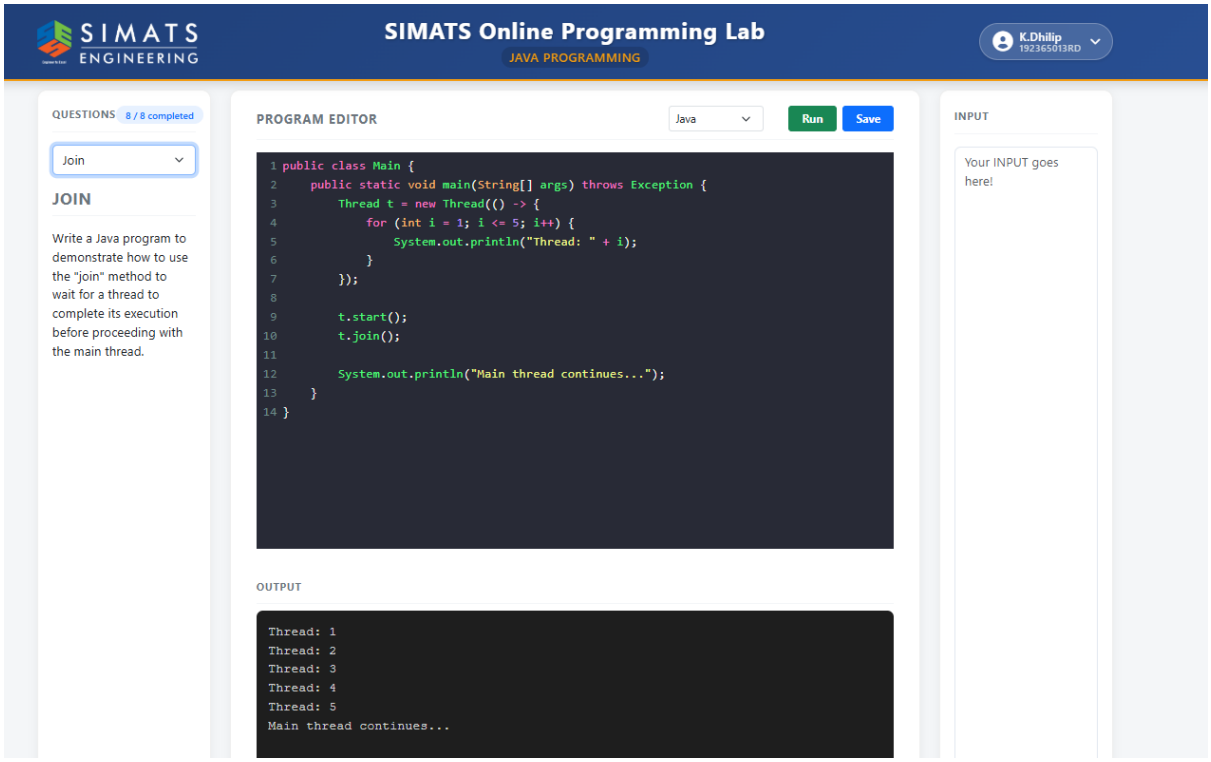
OUTPUT

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

17.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
192365013RD
QUESTIONS 8 / 8 completed
Pool
POOL
Write a Java program to create a thread pool that can execute multiple threads concurrently.
PROGRAM EDITOR
Java
Run
Save
1 import java.util.concurrent.*;
2
3 public class Main {
4 public static void main(String[] args) {
5 ExecutorService pool = Executors.newFixedThreadPool(3);
6
7 for (int i = 1; i <= 5; i++) {
8 int n = i;
9 pool.execute(() -> {
10 System.out.println("Task " + n + " executed by " +
11 Thread.currentThread().getName());
12 });
13 }
14
15 pool.shutdown();
16 }
17 }
INPUT
Your INPUT goes here!
OUTPUT
Task 2 executed by pool-1-thread-2
Task 3 executed by pool-1-thread-3
Task 1 executed by pool-1-thread-1
Task 5 executed by pool-1-thread-3
Task 4 executed by pool-1-thread-2

18.


SIMATS ENGINEERING
SIMATS Online Programming Lab
JAVA PROGRAMMING
K.Dhilip
192365013RD
QUESTIONS 8 / 8 completed
Join
JOIN
Write a Java program to demonstrate how to use the "join" method to wait for a thread to complete its execution before proceeding with the main thread.
PROGRAM EDITOR
Java
Run
Save
1 public class Main {
2 public static void main(String[] args) throws Exception {
3 Thread t = new Thread(() -> {
4 for (int i = 1; i <= 5; i++) {
5 System.out.println("Thread: " + i);
6 }
7 });
8
9 t.start();
10 t.join();
11
12 System.out.println("Main thread continues...");
13 }
14 }
INPUT
Your INPUT goes here!
OUTPUT
Thread: 1
Thread: 2
Thread: 3
Thread: 4
Thread: 5
Main thread continues...